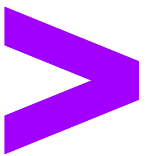


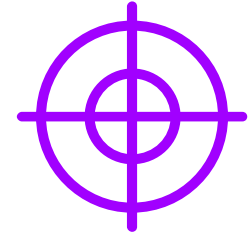
Page Object Model



Module Objectives

At the end of this course, you will be able to:

- Describe Page Object Model (POM) design pattern.
- Explain the process of creating page objects using page factory.



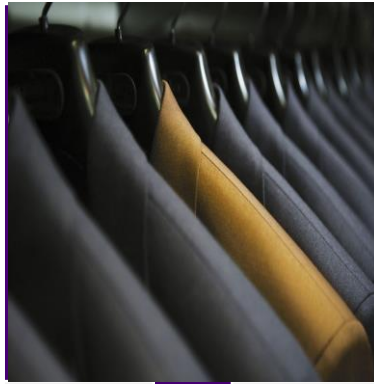
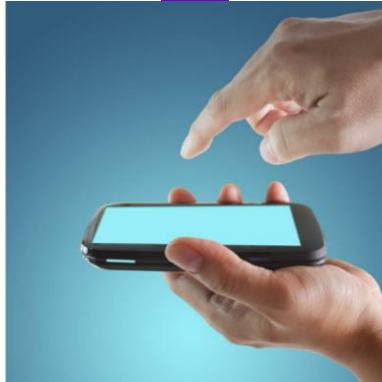
Ground Rules

For a successful class, please:



Arrive on time

Keep your cell phones off



Wear business formals

Assist and show respect to your colleagues irrespective of skill and knowledge level



Do not use the web for personal reasons during training sessions

Adhere to attendance policy as directed by training coordinator



WebDriver API

- 01** Page Object Model (POM)
- 02** Why Page Object Model
- 03** Advantages of POM
- 04** POM Implementation

Page Object Model (POM)

Page Object Model (1)

POM- Page Object Model

POM is a design pattern used in test automation.

It is the framework in which one can create Object Repository for web UI elements like buttons, links, texts using locator information.

We also provide information about the actions to be done like Sendkeys, Select, click etc.

We need to create the page class for each webpage in the application.

It becomes useful to create abstraction for the webpage called page object

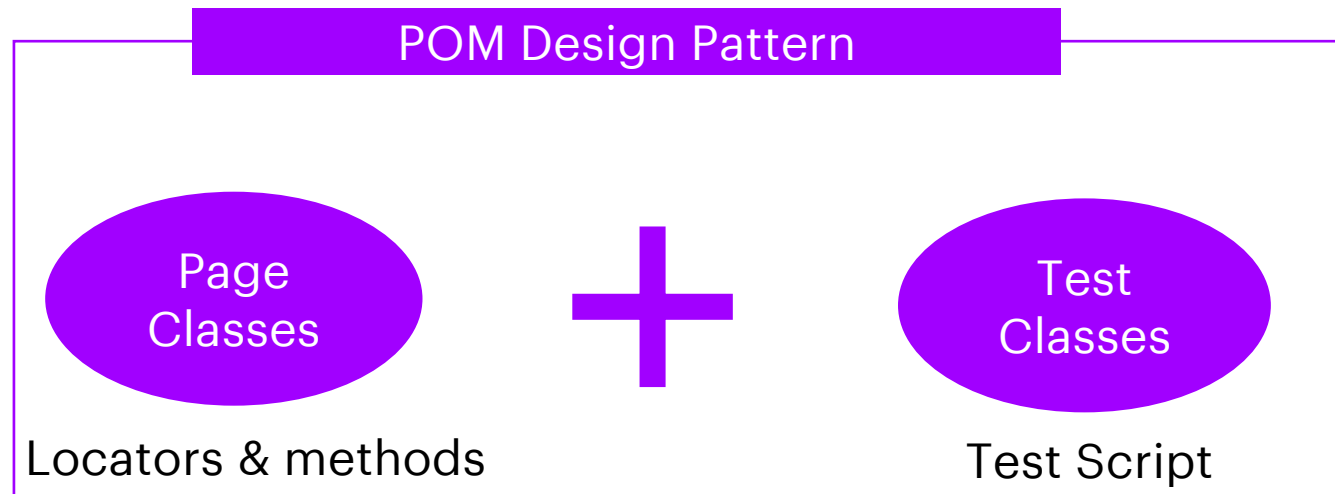
These page objects represent the things being filled in or clicked for examination context.

Page Object Model (2)

What is Page Object Model

For Example:

- Consider a login page object with a method called **“Login”** that includes the different web elements on the web page
- You can create page class and store the web elements in page class for Login webpage.
- This object allows you to easily reuse codes and keep it easy to read.



Why Page Object Model

Why Page Object Model

Reusability

- Consider a Login test script, being small with a smaller number of elements involved, is easy to maintain
- The problem arises in script maintenance when many test scripts are using the same web page elements. We need to change the elements locator in all the test scripts, in case there is a change in UI elements locator by the development team. This creates a lot of rework, affecting the productivity of the team
- By using the page object model; we need to make changes in only a few page classes, instead of many test scripts
- This design pattern makes the code highly reusable and calls for less maintenance

```
Run All
public class Trainingschedule {

    WebDriver driver;
    Sportscoachingpage01 sc;

    @BeforeClass
    public void beforeclass() {
        // Launching browser
        driver = new EdgeDriver();

        // maximizing the browser
        driver.manage().window().maximize();

        // navigating to the URL
        driver.get("https://lkmdemoaut.accenture.com/AccenSportz/#/widgets/sportscoaching");
    }

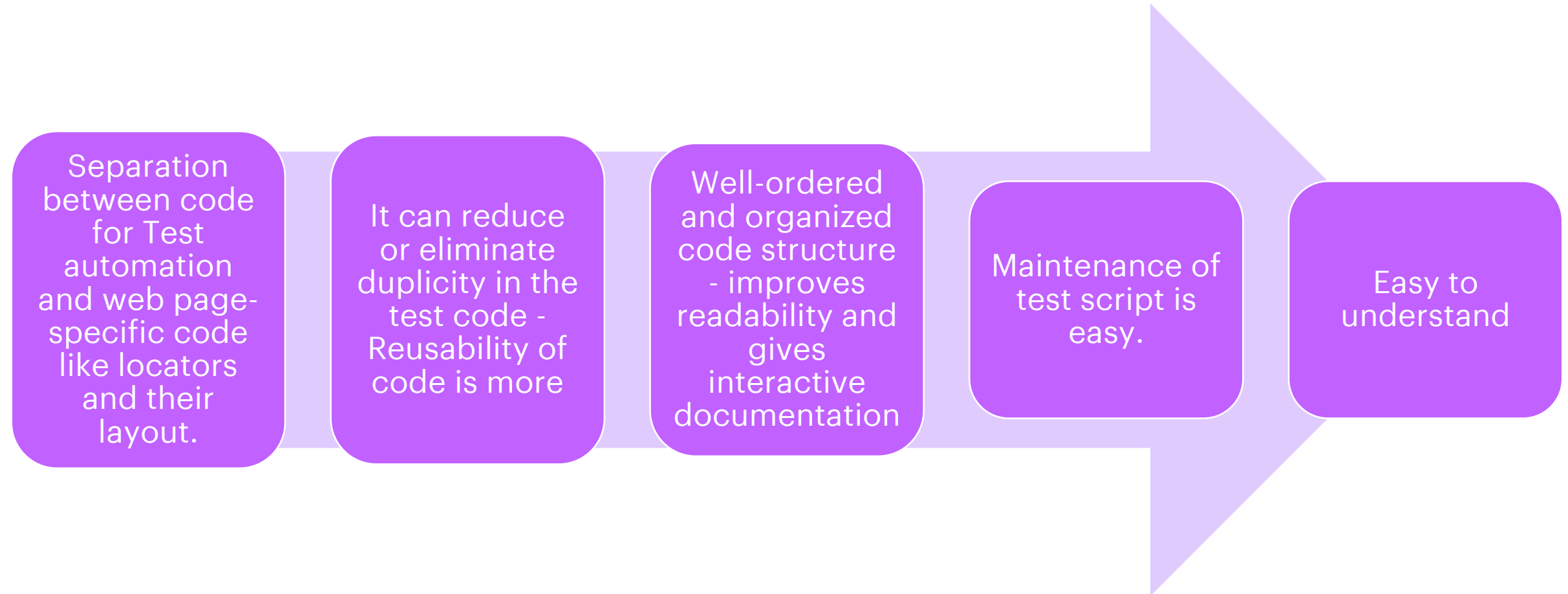
    @AfterClass
    public void afterclass() {
        // closing the browser
        // driver.quit();
    }

    @Test
    Run | Debug
    public void f() {
        sc = new Sportscoachingpage01(driver);
        sc.trainingschedule("02/16/2024");
    }
}
```

Advantages of POM

Advantages of Page object model

Key Advantages

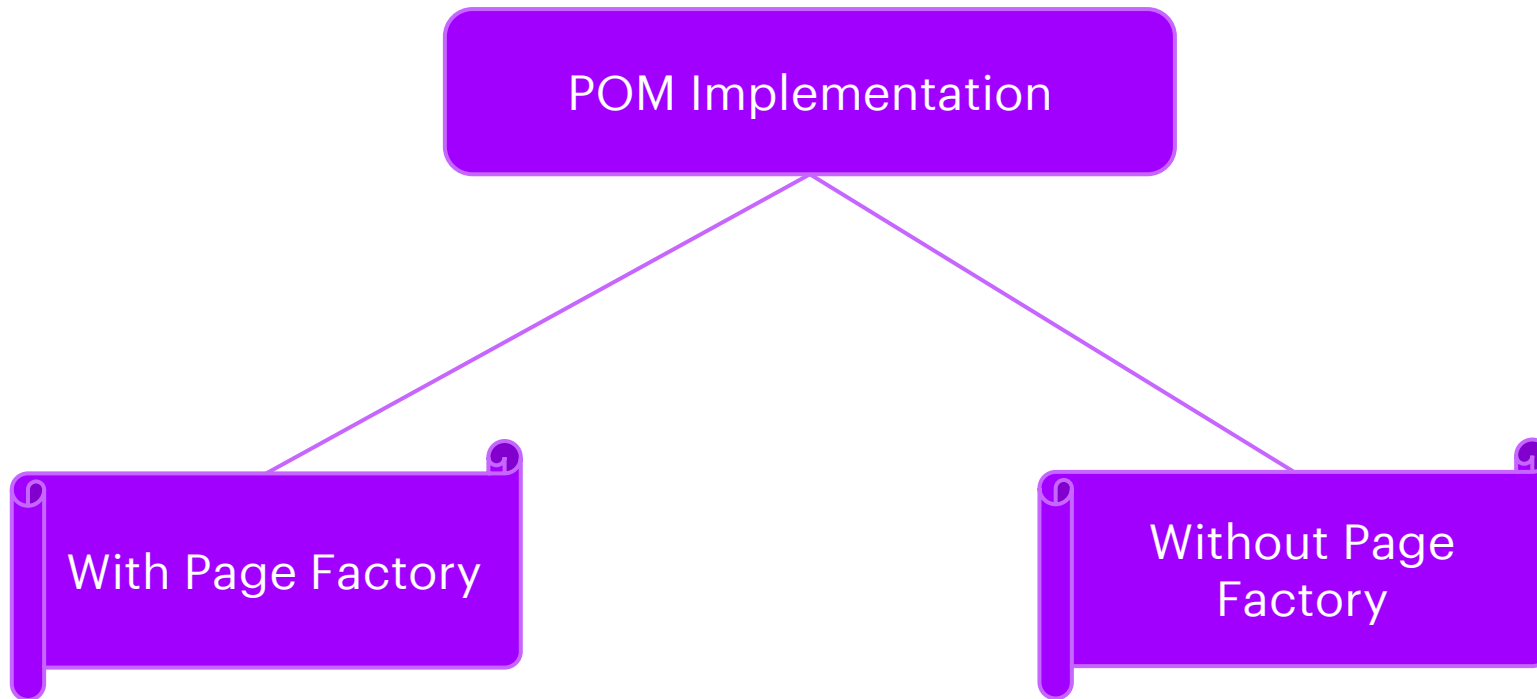


POM Implementation

POM Implementation(1)

Overview of Page Object Model Implementation

- Page Object Model is a design pattern in web test automation to create an Object Repository for web UI elements



POM Implementation(2)

Page Object Model Implementation (without Page Factory)

- A Page Object class is defined for each web page in the test application
- Each Page Object class contains a set of elements locator, a constructor, and methods

- Elements Locator

```
//Elements
By schedule= By.name("trainingschedulebutton");
By bookslot= By.id("bookslot");
By sports= By.id("mat-input-0");
By football= By.xpath("//span[text()=' Football ']");
By date = By.id("Date");
By slottime= By.id("mat-mdc-form-field-label-4");
By time= By.xpath("//span[text()=' 4 PM - 5 PM ']");
By submit= By.name("submitbutton");
By message =By.xpath("//h2[text()='Thank You!']");
```

- Constructor

```
public HomePage(WebDriver driver) {
    this.driver = driver;
}
```

- Methods

```
//methods
public void scheduletraining(String date1) {

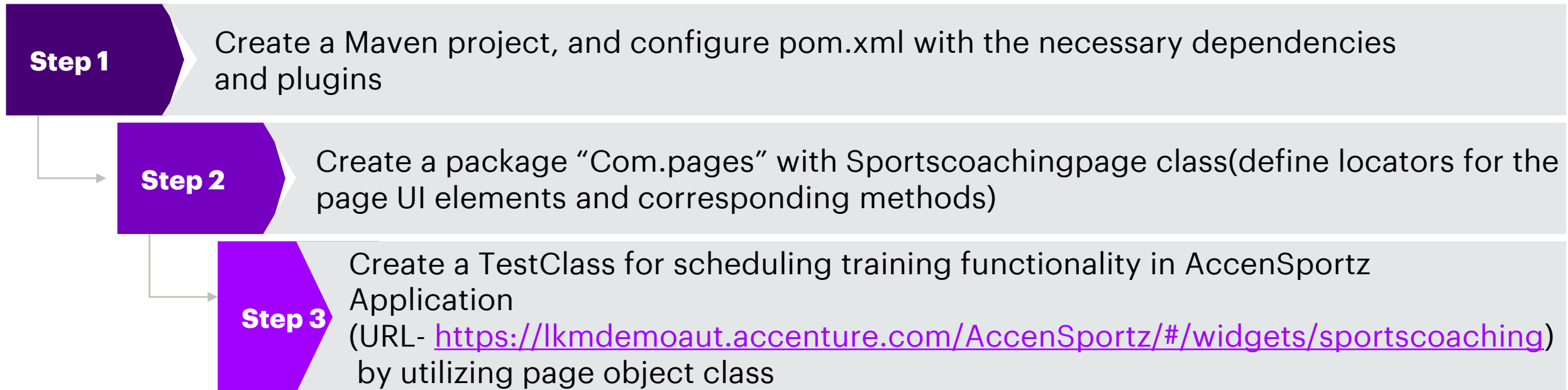
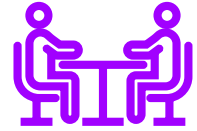
    driver.findElement(schedule).click();
    driver.findElement(bookslot).click();
    driver.findElement(sports).click();
    driver.findElement(football).click();
    driver.findElement(date).sendKeys(date1);
    driver.findElement(slottime).click();
    driver.findElement(time).click();
    driver.findElement(submit).click();

    Assert.assertEquals(driver.findElement(message).getText(), "Thank You!");
}
```

POM Implementation(3)

Demo: Execute Page Object Model Implementation (without Page Factory)

- Linda wants to implement page object model(without page factory)
- Scenario



POM Implementation(4)



Demo Solution: Execute Page Object Model Implementation (without Page Factory)

- ✓  SeleniumProject
 - >  src/main/java
 - >  src/main/resources
 - ✓  src/test/java
 - ✓  Com.pages
 - >  Sportscoachingpage.java
 - >  test
 - >  Util
 - >  src/test/resources
 - >  JRE System Library [JavaSE-1.8]
 - >  Maven Dependencies
 - >  src
 - >  target
 - >  test-output
 -  pom.xml

POM Implementation(5)



Demo Solution: Execute Page Object Model Implementation (without Page Factory)

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.testng.Assert;

public class Sportscoachingpage {

    WebDriver driver;

    //Elements
    By schedule= By.name("trainingschedulebutton");
    By bookslot= By.id("bookslot");
    By sports= By.id("mat-input-0");
    By football= By.xpath("//span[text()=' Football ']");
    By date = By.id("Date");
    By slottime= By.id("mat-mdc-form-field-label-4");
    By time= By.xpath("//span[text()=' 4 PM - 5 PM ']");
    By submit= By.name("submitbutton");
    By message =By.xpath("//h2[text()='Thank You!']");

    //constructor
    public Sportscoachingpage(WebDriver driver) {

        this.driver= driver;
    }

    //methods
    public void scheduletraining(String datel) {

        driver.findElement(schedule).click();
        driver.findElement(bookslot).click();
        driver.findElement(sports).click();
        driver.findElement(football).click();
        driver.findElement(date).sendKeys(datel);
        driver.findElement(slottime).click();
        driver.findElement(time).click();
        driver.findElement(submit).click();

        Assert.assertEquals(driver.findElement(message).getText(), "Thank You!");
    }
}
```

POM Implementation(6)



Demo Solution: Execute Page Object Model Implementation (without Page Factory)

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.edge.EdgeDriver;
import org.testng.annotations.AfterClass;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;
import Com.pages.Sportscoachingpage;
```

Run All

```
public class Trainingschedule {

    WebDriver driver;
    Sportscoachingpage sc;

    @BeforeClass
    public void beforeclass() {
        // Launching browser
        driver = new EdgeDriver();

        // maximizing the browser
        driver.manage().window().maximize();

        // navigating to the URL
        driver.get("https://lkmdemoaut.accenture.com/AccenSportz/#/widgets/sportscoaching");
    }

    @AfterClass
    public void afterclass() {
        // closing the browser
        // driver.quit();
    }
}
```

@Test

Run | Debug

```
public void f() {
    sc = new Sportscoachingpage(driver);
    sc.scheduletraining("02/16/2024");
}
```

POM Implementation(7)



Demo Result: Execute Page Object Model Implementation (without Page Factory)

```
[RemoteTestNG] detected TestNG version 7.9.0
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Oct 29, 2024 10:50:54 AM org.openqa.selenium.manager.SeleniumManager lambda$runComm
WARNING: Error sending stats to Plausible: error sending request for url (https://p
Oct 29, 2024 10:51:04 AM org.openqa.selenium.devtools.CdpVersionFinder findNearestM
WARNING: Unable to find an exact match for CDP version 130, returning the closest v
PASSED: test.Trainingschedule.f
```

```
=====
Default test
Tests run: 1, Failures: 0, Skips: 0
=====
```

```
=====
Default suite
Total tests run: 1, Passes: 1, Failures: 0, Skips: 0
=====
```

POM Implementation(8)

Page Object Model Implementation (with Page Factory)

Page factory is a factory class in selenium library to load all the page objects.

List<WebElement> fields are identified using **@FindBy** or **@FindBy** annotation.

In Page factory *initElements()* is used to initialize the objects from page class.

Different methods should be created for different results of same functionality.

In Page Objects, there is no need to represent the entire page; the required element is only presented.

POM Implementation(9)

Page Object Model Implementation (with Page Factory)

- Page factory is a factory class in selenium library, which is used to load all the page objects
- List<WebElement> fields are identified **@FindBy** or **@FindBy** annotation
- **@FindBy** can accept all the locators like name, id, linkText, etc.
- **@FindBy** is used to spot an element on a Page Object for indicating different instrument for finding an element or a list of elements. It can be used with 'how' and 'using' keyword
- For example, these two annotations indicate the same element:
 - **@FindBy(name="userName")**
WebElement uname;
 - **@FindBy(how=How.NAME, using="userName")**
WebElement uname;

POM Implementation(10)

Overview of Page Object Model Implementation

- A Page Object class is defined for each web page in the test application
- Each Page Object class contains a set of elements locator, a constructor, and methods

- Elements Locator



```
//Elements
@FindBy(name ="trainingschedulebutton")
WebElement Schedule;
```

- Constructor



```
public HomePage(WebDriver driver) {
    this.driver = driver;
    PageFactory.initElements(driver, this);
}
```

- Methods

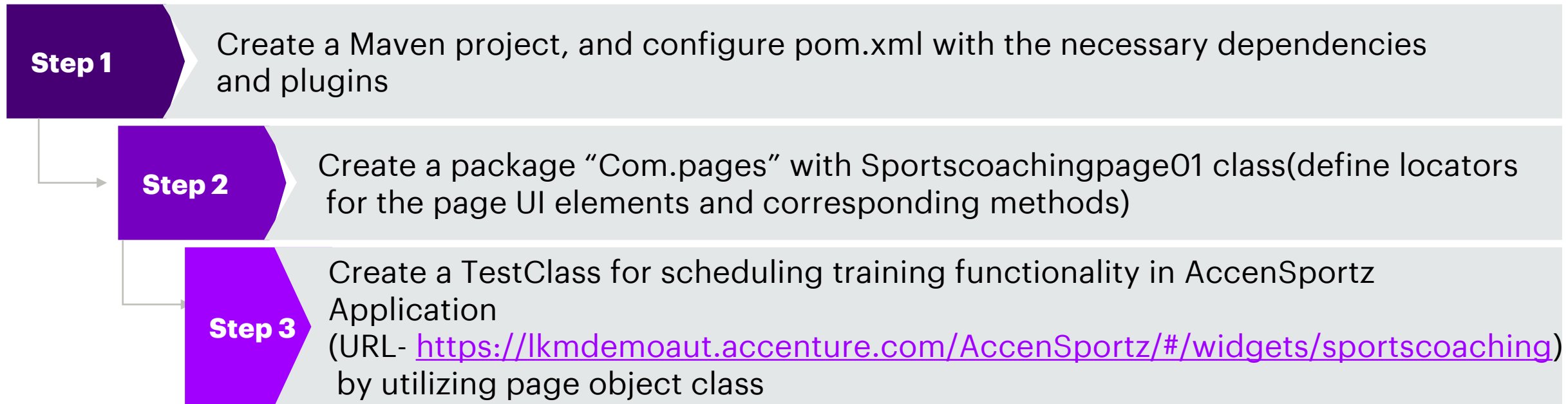
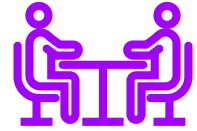


```
//Methods
public void trainingschedule(String date1) {
    Schedule.click();
    bookslot.click();
    sports.click();
    football.click();
    date.sendKeys(date1);
    slottime.click();
    time.click();
    submit.click();
    Assert.assertEquals(message.getText(), "Thank You!");
}
```

POM Implementation(11)

Demo: Execute Page Object Model Implementation (with Page Factory)

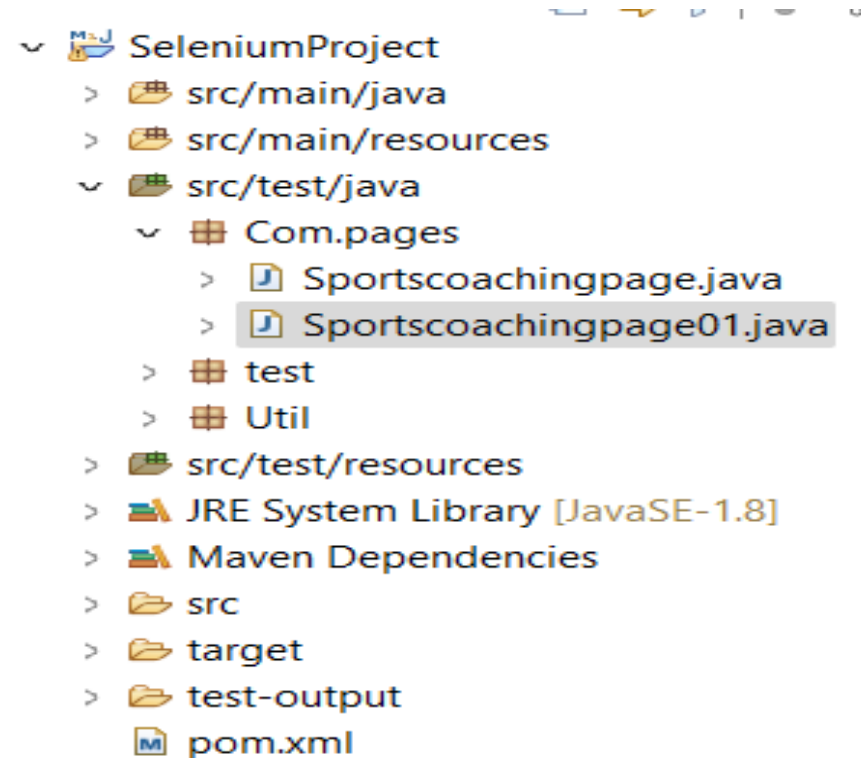
- Linda wants to implement Page object model(with page factory)
- Scenario



POM Implementation(12)



Demo Solution: Execute Page Object Model Implementation (with Page Factory)



POM Implementation(13)



Demo Solution: Execute Page Object Model Implementation (with Page Factory)

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.PageFactory;
import org.testng.Assert;

public class Sportscoachingpage01 {

    WebDriver driver;

    //Elements
    @FindBy(name = "trainingschedulebutton")
    WebElement Schedule;
    @FindBy(id = "bookslot")
    WebElement bookslot;
    @FindBy(id = "mat-input-0")
    WebElement sports;
    @FindBy(xpath = "//span[text()=' Football ']")
    WebElement football;
    @FindBy(id = "Date")
    WebElement date;
    @FindBy(id = "mat-mdc-form-field-label-4")
    WebElement slottime;
    @FindBy(xpath = "//span[text()=' 4 PM - 5 PM ']")
    WebElement time;
    @FindBy(name = "submitbutton")
    WebElement submit;
    @FindBy(xpath = "//h2[text()='Thank You!']")
    WebElement message;

    //constructor
    public Sportscoachingpage01(WebDriver driver) {
        this.driver=driver;
        PageFactory.initElements(driver, this);
    }
}
```

```
//Methods
public void trainingschedule(String date1) {
    Schedule.click();
    bookslot.click();
    sports.click();
    football.click();
    date.sendKeys(date1);
    slottime.click();
    time.click();
    submit.click();
    Assert.assertEquals(message.getText(), "Thank You!");
}
```

POM Implementation(14)



Demo Solution: Execute Page Object Model Implementation (with Page Factory)

```
package test;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.edge.EdgeDriver;
import org.testng.annotations.AfterClass;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;
import Com.pages.Sportscoachingpage01;

Run All
public class Trainingschedule {

    WebDriver driver;
    Sportscoachingpage01 sc;

    @BeforeClass
    public void beforeclass() {
        // Launching browser
        driver = new EdgeDriver();

        // maximizing the browser
        driver.manage().window().maximize();

        // navigating to the URL
        driver.get("https://lkmdemoaut.accenture.com/AccenSportz/#/widgets/sportscoaching");
    }

    @AfterClass
    public void afterclass() {
        // closing the browser
        // driver.quit();
    }
}
```

```
@Test
Run | Debug
public void f() {
    sc = new Sportscoachingpage01(driver);
    sc.trainingschedule("02/16/2024");
}
```

POM Implementation(15)



Demo Result: Execute Page Object Model Implementation (with Page Factory)

```
[RemoteTestNG] detected TestNG version 7.9.0
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for fu:
Oct 29, 2024 11:29:52 AM org.openqa.selenium.devtools.CdpVersionFind
WARNING: Unable to find an exact match for CDP version 130, returnin
PASSED: test.Trainingschedule.f
```

```
=====
      Default test
      Tests run: 1, Failures: 0, Skips: 0
=====
```

```
=====
Default suite
Total tests run: 1, Passes: 1, Failures: 0, Skips: 0
=====
```

Knowledge Check



Question 01

Select the variation which locates elements by the value of the “name” attribute in POM.

a) `@FindBy(name="Email")`
`WebElement emailId;`

b) `@FindBy(how=How.NAME,using="Email")`
`WebElement emailId;`

c) Both a and b

d) None of the above

Knowledge Check



Question 02

Which of these is a benefit of using Page Object Model in test automation?

- a) It can reduce or eliminate duplicity in the test code, bringing in high code reusability
- b) It makes test code maintenance easy with very less effort
- c) It enhances readability and gives interactive documentation
- d) All of the above

Module Summary

Now, you will be able to:

- Describe Page Object Model (POM) design pattern.
- Explain the process of creating page objects using page factory



Thank You

